



School of Engineering and Technology

Main Campus, Off Hennur-Bagalur Main Road, Chagalahatti, Bengaluru-562149

MINI PROJECT REPORT

“BUGBOUNTY”

A Full-Stack Vulnerability Disclosure & Bug Bounty Platform

submitted to,

School of Engineering and Technology, CMR University

in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology,

in

COMPUTER SCIENCE AND ENGINEERING

by

MAHA AKSHAY (USN: 24BBTCS352)

SUNDARA KUNDANA MALAVATHU (USN: 24BBTCS341)

KRITESH KUMAR (USN: 24BBTCS351)

Under the guidance of,

Prof. Gripsy Paul

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CMR UNIVERSITY

2026-27

School of Engineering and Technology, CMR

Main Campus, Off Hennur-Bagalur Main Road, Chagalahatti, Bengaluru-562149

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the mini project titled “**BUGBOUNTY**” carried out by MAHA AKSHAY (24BBTCS352), SUNDARA KUNDANA MALAVATHU (24BBTCS341), KRITESH KUMAR (24BBTCS351) in partial fulfilment for the award of Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING of CMR University, during the year 2026-27. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The mini project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Prof Gripsy Paul
Signature of the Guide

Examiners

Name of the examiners	Signature with date
1.	
2.	

DECLARATION

We, MAHA AKSHAY, SUNDARA KUNDANA MALAVATHU, KRITESH KUMAR, students of School of Engineering and Technology, CMR university, hereby declare that the dissertation titled “**BUG BOUNTY**” embodies the report of my mini project carried out independently by us during fifth semester of Bachelor of Technology in Computer Science and Engineering, under the supervision of **Prof. Gripsy Paul**, Department of Computer Science and Engineering and this work has been submitted in partial fulfilment for the award of the Bachelor of Technology degree. We have not submitted the project for the award of any other degree of any other university or institution.

Date :

MAHA AKSHAY (24BBTCS352)

Place :

SUNDARA KUNDANA MALAVATHU

(24BBTCS341)

KRITESH KUMAR (24BBTCS351)

1. Abstract

BugBounty is a full-stack web platform designed to manage vulnerability disclosure and bug bounty activities between security researchers and organizations. Traditional vulnerability reporting through email, forms, or disconnected tools can create duplicate submissions, inconsistent severity decisions, delayed communication, and difficulty in tracking the complete lifecycle of a report. The proposed system provides a centralized platform where researchers can discover bounty programs, review their scope and rules, submit vulnerability reports with proof-of-concept evidence, and track their findings. Administrators can create programs, define in-scope and out-of-scope assets, review reports, manage severity, assign rewards, and monitor triage activities.

The system uses the MERN stack, consisting of MongoDB, Express.js, React, and Node.js, with Socket.IO for real-time communication. It includes an automated risk-scoring mechanism based on CVSS-style metrics, an NLP-based duplicate detection mechanism using tokenization and Jaccard similarity, and a finite state machine to control valid report transitions. Security measures include JWT authentication, bcrypt password hashing, role-based access control, IDOR protection, request rate limiting, input sanitization, secure evidence uploads, and security headers. The project also supports a self-contained local database configuration and automated testing.

The implementation was verified using 74 automated tests covering authentication, bounty programs, vulnerability reports, workflow management, end-to-end functionality, and security boundaries, with all 74 tests passing. The platform therefore provides a structured, transparent, and secure approach to vulnerability disclosure and triage management.

Keywords: Bug Bounty, Vulnerability Disclosure, MERN Stack, CVSS, NLP, Jaccard Similarity, FSM, Cybersecurity, React, Node.js

2. Introduction

Modern web applications and distributed software systems are exposed to a wide range of security threats. Security teams use security reviews and automated scanners to identify vulnerabilities, but some issues are difficult to detect automatically. These can include business logic weaknesses, authorization problems, multi-step attack paths, and other application-specific security flaws. Bug bounty programs provide an additional security layer by allowing independent ethical security researchers to find and report vulnerabilities to organizations.

Although bug bounty programs can improve security, managing large numbers of vulnerability reports can become difficult. Organizations may receive many low-quality, duplicate, or out-of-scope submissions. Analysts may also have to manually decide the severity and priority of reports. When the process is handled through email or disconnected tools, tracking report status, communication, rewards, and audit history becomes more difficult.

BugBounty addresses these challenges through a centralized vulnerability disclosure and triage management platform. The platform separates the responsibilities of two major user roles: the security researcher and the administrator. Researchers can browse active programs, understand their scope, submit reports, attach evidence, communicate with analysts, and view rewards and reputation. Administrators can create programs, define targets and rules, review submitted findings, control the triage process, change severity, authorize rewards, and view analytics.

The project follows a layered MERN architecture. React provides the client-side interface, Express.js provides REST API routing, Node.js manages the server-side application logic, and MongoDB provides data persistence. Socket.IO is used to provide real-time notifications. The business logic includes automated risk scoring, duplicate detection, controlled workflow transitions, reputation management, and notifications.

The project is also designed with security in mind because the platform itself handles sensitive vulnerability information. Authentication, authorization, IDOR protection, password hashing, input sanitization, rate limiting, secure file upload validation, and security headers are included in the implementation. The report describes the problem, objectives, system analysis, design, implementation, results, and future enhancements of the BugBounty platform.

3. Problem Statement

Organizations running vulnerability disclosure or bug bounty programs may face a large number of vulnerability reports, including duplicate, low-quality, or out-of-scope submissions. Manual triage can result in inconsistent risk prioritization, increased analyst workload, delayed responses, weak tracking of report status, and difficulty in maintaining a complete audit trail. Researchers may also experience poor visibility into report progress and reward status. Therefore, there is a need for a centralized, secure, and structured platform that automates important parts of vulnerability intake, risk assessment, duplicate detection, triage workflow, communication, and reward management.

4. Objectives

- To develop a centralized platform for vulnerability disclosure and bug bounty management.
- To provide separate and secure functionality for security researchers and administrators.
- To automatically calculate a risk score using defined vulnerability and exploitability factors.
- To detect potentially duplicate vulnerability reports using NLP and Jaccard similarity.
- To enforce valid report status transitions using a finite state machine.
- To provide secure authentication, role-based authorization, and protection against unauthorized data access.
- To allow researchers and administrators to communicate through report comments and notifications.
- To manage bounty rewards and researcher reputation in a structured manner.
- To provide real-time status and reward notifications using Socket.IO.
- To test and verify the major functional and security components of the system.

5. System Analysis

5.1 Existing System

In a basic vulnerability disclosure process, researchers may submit findings through email, web forms, or separate ticketing systems. The security team then manually reads each report, checks whether it is in scope, determines severity, searches for duplicates, communicates with the researcher, and records the final decision. This approach can work for a small number of reports but becomes inefficient as the number of submissions increases.

- Duplicate reports may require repeated manual investigation.
- Severity and priority may be inconsistent when there is no standardized scoring process.
- Report status and audit history may be difficult to track.
- Communication between researchers and analysts may be delayed.
- Sensitive vulnerability information may be spread across multiple systems.
- Reward and reputation management may require additional manual work.

5.2 Proposed System

BugBounty provides a centralized MERN-based platform for the complete vulnerability disclosure lifecycle. Researchers can access bounty programs and submit vulnerability reports with evidence. The system automatically performs risk assessment and duplicate checking. Administrators can review and triage findings through controlled workflow states, communicate with researchers, assign rewards, and monitor analytics. Real-time notifications keep users informed about important changes.

- Centralized management of programs, reports, comments, rewards, and notifications.
- Automated CVSS-style risk scoring for consistent prioritization.
- NLP-based multi-vector duplicate detection.
- Finite state machine enforcement for report lifecycle management.
- Role-based access control with strict ownership checks.
- Secure evidence upload validation and protected internal comments.
- Real-time notifications using Socket.IO.
- Local database operation for self-contained development and testing.

5.3 Hardware Requirements

- **Processor:** Modern dual-core or better processor.
- **RAM:** Minimum 4 GB; 8 GB or more recommended for development.
- **Storage:** At least 5 GB free space for source code, dependencies, database, and evidence files.

-
- **Network:** Internet connection may be used for package installation and development, while the embedded local database can operate without a cloud database.

5.4 Software Requirements

- **Operating System:** Windows, Linux, or macOS.
- **Frontend:** React 18, Vite 6, Tailwind CSS, Lucide React, and Recharts.
- **Backend:** Node.js 20 and Express.js 4.
- **Database:** MongoDB with WiredTiger and Mongoose 8.
- **Real-Time Communication:** Socket.IO.
- **Authentication and Security:** JWT, bcrypt, rate limiting, input sanitization, and Helmet.
- **Testing:** Jest and Supertest.
- **Development Tools:** npm and a suitable code editor.

6. System Design

6.1 System Architecture

The BugBounty system follows a layered architecture. The presentation layer is implemented using React and provides role-aware pages, forms, dashboards, and visualizations. The API layer uses Express.js to handle REST requests, authentication, authorization, validation, and routing. The service layer contains the main business logic, including risk scoring, duplicate detection, workflow control, reputation calculation, and notification handling. MongoDB provides persistence, while Socket.IO provides real-time communication.

Architecture flow: Researcher/Admin → React Client → Express REST API → Node.js Services → MongoDB. Socket.IO provides a real-time communication path between the server and authenticated clients.

6.2 Data Flow Diagram (DFD)

Level 0 DFD: Researcher/Admin → BugBounty Platform → Database. The researcher submits reports and receives report status, notifications, rewards, and reputation information. The administrator manages programs, reviews reports, changes status/severity, and assigns rewards. The platform validates requests, processes business rules, stores data, and sends notifications.

Level 1 DFD modules: (1) Authentication and RBAC, (2) Program Management, (3) Vulnerability Report Management, (4) Risk and Duplicate Analysis, (5) Triage Workflow, (6) Comments and Notifications, (7) Reward and Reputation Management, and (8) Analytics.

6.3 Use Case Diagram

Primary actors are Security Researcher and Administrator. The Security Researcher can register/login, view active bounty programs, view program scope and rules, submit reports, upload evidence, view report status, comment on reports, view rewards, view notifications, and view leaderboard information. The Administrator can authenticate, create/update programs, view all reports, review and triage findings, change severity, manage comments, authorize rewards, view analytics, and manage the overall disclosure workflow.

6.4 Entity Relationship Design

The main data entities are User, BountyProgram, VulnerabilityReport, Comment, Reward, Notification, and ReputationLog. A User can create or participate in reports and can be associated with comments, rewards, notifications, and reputation records. A BountyProgram is created by a user with administrative privileges and can contain multiple VulnerabilityReports.

Each VulnerabilityReport is linked to a program and researcher and may have comments, evidence, status history, timeline events, and a reward.

6.5 Workflow / Algorithm

The main report workflow is: Submitted → Under Review → Triaged → Accepted → Reward Assigned → Resolved. A report can also move from submitted, under review, or triaged to Rejected. A rejected report may return to Under Review when a valid appeal is submitted. Illegal transitions are rejected by the workflow service. Risk scoring is performed during report processing, and duplicate detection compares the affected asset, title, technical content, and vulnerability category.

7. Implementation

7.1 Technologies Used

The project is implemented using the MERN stack. React 18 and Vite provide the frontend single-page application. Node.js and Express.js provide the backend API. MongoDB with Mongoose provides database storage. Socket.IO provides real-time communication. Tailwind CSS supports interface styling and Recharts supports data visualization.

7.2 Authentication and Authorization

The platform uses JWT-based authentication. Passwords are protected using bcrypt with 12 salt rounds. Public registration defaults to the researcher role. Administrator registration requires a server-side administrative secret. Middleware checks the authenticated user's identity and role before protected operations are allowed.

7.3 Vulnerability Reporting

Researchers submit vulnerability information including the title, category, affected asset, reproduction steps, impact, severity information, and supporting evidence. The platform performs risk assessment and duplicate checking when the report is processed. Researchers can later track the status and communicate through comments.

7.4 Risk Scoring

The system implements a deterministic CVSS-style scoring mechanism with factors such as impact, exploitability, attack vector, data exposure, and authentication requirements. The resulting score is normalized to a range of 0.0 to 10.0 and mapped to risk bands: Critical, High, Medium, Low, and None.

7.5 Duplicate Detection

Duplicate detection compares four main vectors: target asset, vulnerability title, technical content, and vulnerability category. Titles and technical descriptions are normalized and tokenized. Stopwords are removed and Jaccard similarity is used to measure lexical overlap. Asset and category matches provide additional evidence when determining whether a submission resembles an existing report.

7.6 Triage Workflow

The finite state machine prevents invalid changes to report status. For example, a reward cannot be assigned before a report reaches an accepted state. Each valid transition is recorded in the report history and timeline. This provides traceability for the triage process.

7.7 Security Implementation

- JWT authentication and role-based access control.
- bcrypt password hashing with 12 salt rounds.
- Strict ownership checks to prevent IDOR.
- MongoDB operator sanitization against NoSQL injection.
- Helmet security headers.
- Rate limiting on authentication endpoints.
- File type and extension validation for evidence uploads.
- Internal comment filtering so confidential administrator notes are not exposed to researchers.

7.8 Real-Time Notifications

Socket.IO is used for bi-directional communication between the server and authenticated users. Notifications can be delivered when report status changes or rewards are assigned. This reduces the need for users to repeatedly refresh the application.

8. Results and Discussion

The BugBounty platform successfully implements the major functions required for vulnerability disclosure and triage management. The system supports separate researcher and administrator workflows, automated risk assessment, duplicate detection, controlled triage states, rewards, reputation, notifications, and security controls.

8.1 Testing Results

The project includes 74 automated tests using Jest and Supertest. The tests cover authentication, bounty program management, vulnerability report processing, evidence handling, risk scoring, duplicate detection, workflow transitions, reward allocation, end-to-end lifecycle behavior, IDOR protection, NoSQL injection defense, and privilege escalation prevention.

Overall result: 74 out of 74 tests passed, giving a 100% pass rate for the reported automated test suite.

8.2 Key Results

- Authentication and role controls were verified.
- Bounty program creation, scope validation, and visibility rules were tested.
- Vulnerability report ingestion and evidence handling were tested.
- Risk scoring and risk-band assignment were tested.
- Duplicate similarity scoring was tested.
- Finite state machine transitions and illegal transition blocking were tested.
- Cross-researcher access was blocked through IDOR protection.
- Security defenses against NoSQL injection and privilege escalation were verified.
- Reward allocation and reputation updates were tested.

8.3 Screenshots

The implemented interfaces and administrative triage workflows of the BugBounty platform are shown below:

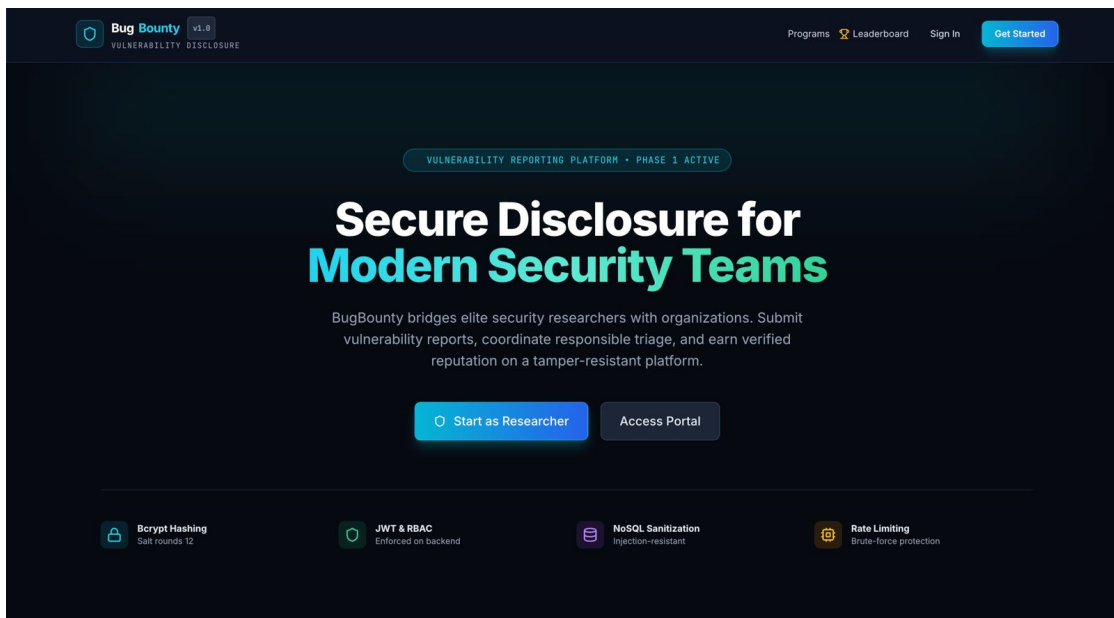


Figure 8.1: BugBounty Landing Page and Platform Overview

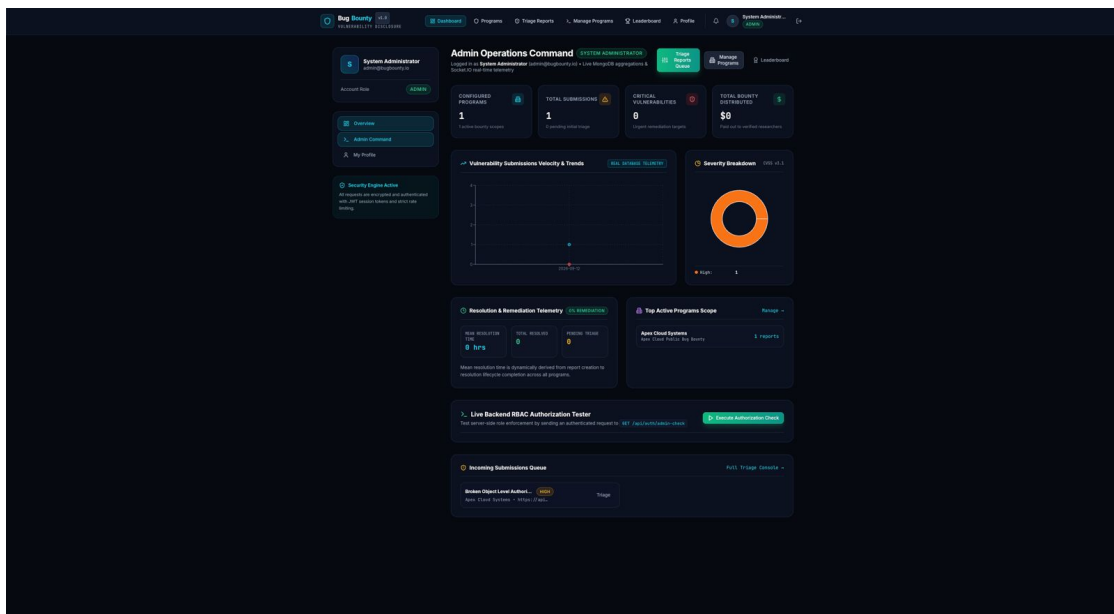


Figure 8.2: Admin Operations Command Dashboard and Telemetry

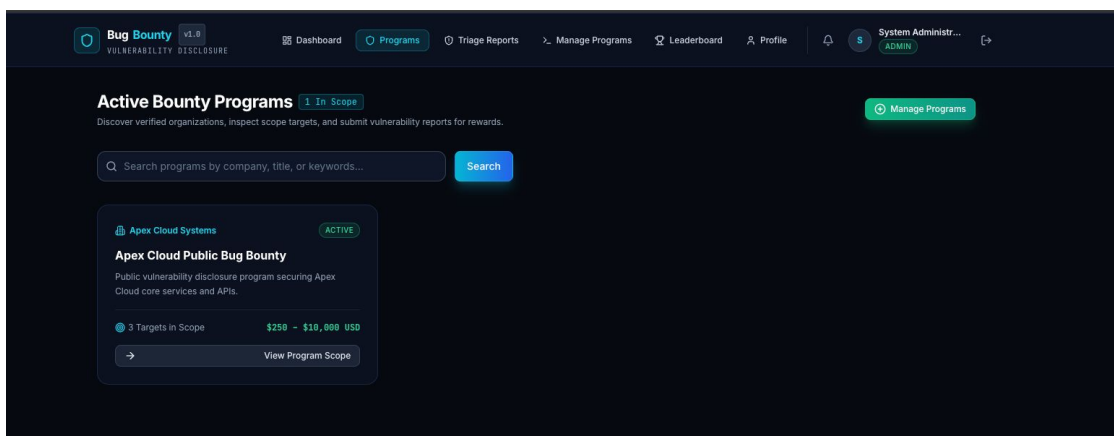


Figure 8.3: Active Bounty Programs and Scope Discovery

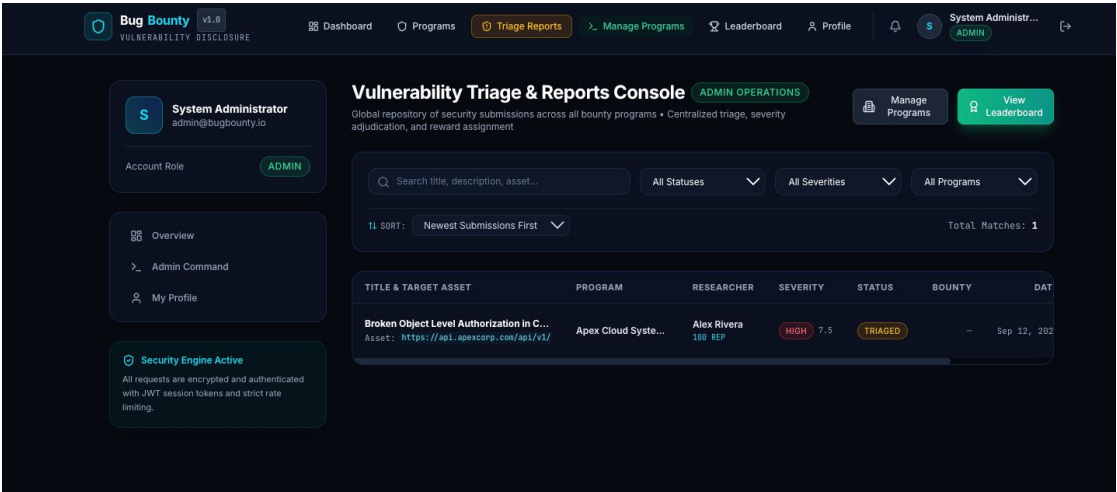


Figure 8.4: Vulnerability Triage and Reports Console

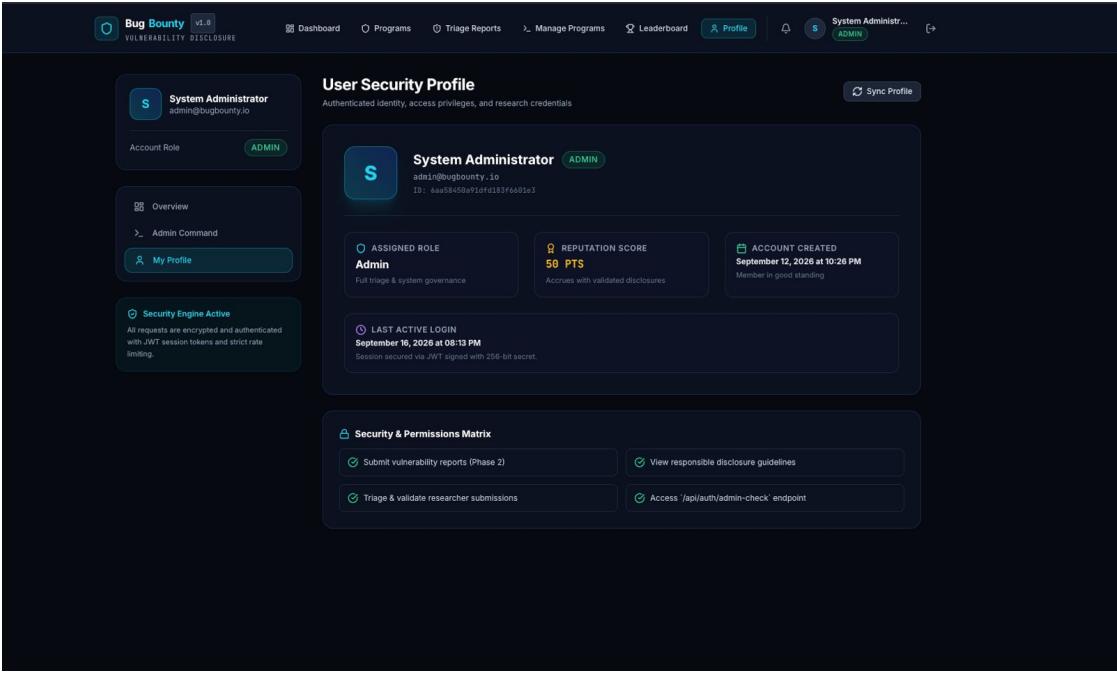


Figure 8.5: User Security Profile and Role Governance

9. Conclusion and Future Work

The BugBounty project provides a centralized and structured solution for vulnerability disclosure and bug bounty triage. It addresses important weaknesses in manual and fragmented vulnerability reporting by combining report management, automated risk scoring, duplicate detection, controlled workflow states, secure communication, rewards, reputation, and notifications in one platform.

The MERN architecture provides a clear separation between the frontend, backend, business logic, and data persistence layers. Security mechanisms such as JWT authentication, bcrypt password hashing, RBAC, IDOR protection, NoSQL injection defense, rate limiting, secure file validation, and security headers strengthen the platform. The reported automated verification also demonstrates strong functional and security coverage, with 74 of 74 tests passing.

9.1 Future Work

- AI-powered remediation advice could be added to provide developers with contextual suggestions for fixing accepted vulnerabilities.
- Enterprise SAML/SSO integrations could be added for organizational authentication.
- Automated payout gateways could be integrated for direct bounty disbursement.
- Public redacted vulnerability disclosure could be supported after vulnerabilities are patched.
- The platform could be expanded with additional analytics and integrations for larger security operations teams.